```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os
import shutil
import cv2
import numpy as np
from tqdm import tqdm
from sklearn.model_selection import train_test_split, KFold
from sklearn.utils import shuffle
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Dropout
import seaborn as sns
import matplotlib.pyplot as plt
import tensorflow as tf


dataset_folder = "/content/drive/MyDrive/ALzh"
output_train_folder = "/content/drive/MyDrive/RESULT/Train"
output_test_folder = "/content/drive/MyDrive/RESULT/Test"
labels = ['AD updated', 'CN Updated', 'MCI Updated']

os.makedirs(output_train_folder, exist_ok=True)
os.makedirs(output_test_folder, exist_ok=True)

for label in labels:
    section_path = os.path.join(dataset_folder, label)
    if not os.path.isdir(section_path):
        print(f"Directory not found for label: {label}. Skipping...")
        continue

    files = [f for f in os.listdir(section_path) if os.path.isfile(os.path.join(section_path, f))]
    if not files:
        print(f"No files found in section: {label}. Skipping...")
        continue

    train_files, test_files = train_test_split(files, test_size=0.2, random_state=42)

    train_section_path = os.path.join(output_train_folder, label)
    test_section_path = os.path.join(output_test_folder, label)
    os.makedirs(train_section_path, exist_ok=True)
    os.makedirs(test_section_path, exist_ok=True)

    for file in train_files:
        shutil.copy(os.path.join(section_path, file), os.path.join(train_section_path, file))
    for file in test_files:
        shutil.copy(os.path.join(section_path, file), os.path.join(test_section_path, file))

    print(f"Section: {label}")
    print(f"Total: {len(files)}, Train: {len(train_files)}, Test: {len(test_files)}")
    print("-" * 50)

print("Dataset successfully split into training and testing sets!")
```

```
Section: AD updated
Total: 534, Train: 427, Test: 107
--------------------------------------------------
Section: CN Updated
Total: 913, Train: 730, Test: 183
--------------------------------------------------
Section: MCI Updated
Total: 894, Train: 715, Test: 179
--------------------------------------------------
Dataset successfully split into training and testing sets!
```

```
def load_images(folder_path, label, data_list, label_list):
    for image_file in tqdm(os.listdir(folder_path), desc=f"Loading {label}"):
        image_path = os.path.join(folder_path, image_file)
        image = cv2.imread(image_path)
        if image is not None:
            image = cv2.resize(image, (160, 160))
            data_list.append(image)
            label_list.append(label)

image_size = 160
train_path = output_train_folder
```

```
test_path = output_test_folder

X_train, Y_train = [], []
X_test, Y_test = [], []

for label in labels:
    path = os.path.join(train_path, label)
    if os.path.exists(path):
        load_images(path, label, X_train, Y_train)

for label in labels:
    path = os.path.join(test_path, label)
    if os.path.exists(path):
        load_images(path, label, X_test, Y_test)

X_train = np.array(X_train)
Y_train = np.array(Y_train)
X_test = np.array(X_test)
Y_test = np.array(Y_test)

X_train, Y_train = shuffle(X_train, Y_train, random_state=42)

label_to_int = {label: idx for idx, label in enumerate(labels)}
Y_train_int = np.array([label_to_int[label] for label in Y_train])
Y_test_int = np.array([label_to_int[label] for label in Y_test])

Y_train_onehot = to_categorical(Y_train_int, num_classes=len(labels))
Y_test_onehot = to_categorical(Y_test_int, num_classes=len(labels))

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Loading AD updated: 100%|██████████| 427/427 [00:04<00:00, 102.13it/s]
Loading CN Updated: 100%|██████████| 730/730 [00:07<00:00, 92.78it/s]
Loading MCI Updated: 100%|██████████| 715/715 [00:09<00:00, 72.56it/s]
Loading AD updated: 100%|██████████| 107/107 [00:00<00:00, 124.85it/s]
Loading CN Updated: 100%|██████████| 183/183 [00:01<00:00, 106.76it/s]
Loading MCI Updated: 100%|██████████| 179/179 [00:01<00:00, 114.57it/s]
Train shape: (1872, 160, 160, 3)
Test shape: (469, 160, 160, 3)
```

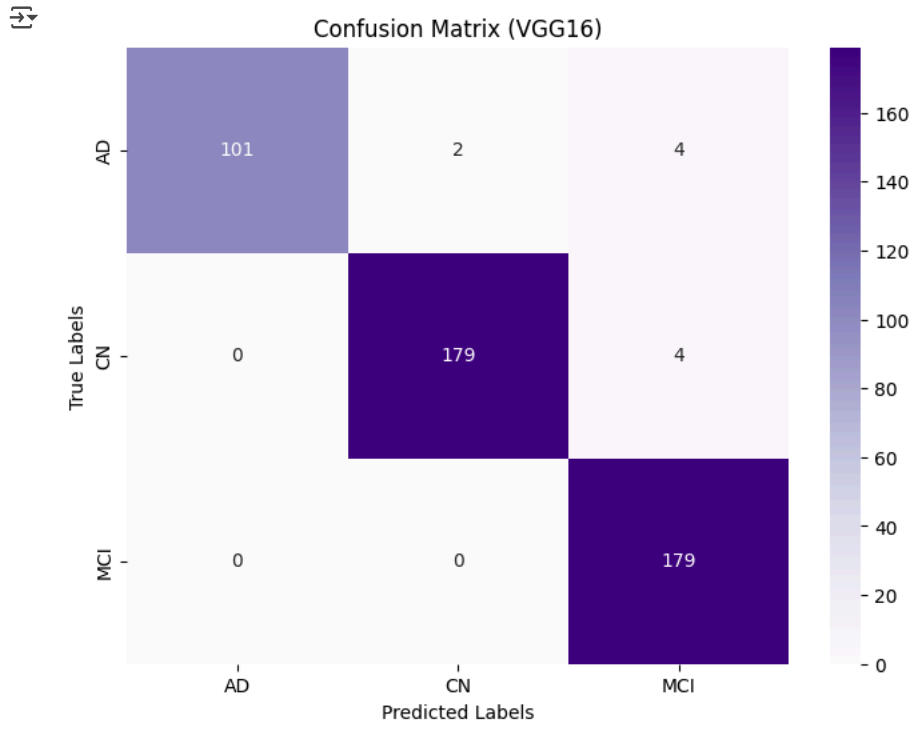
```
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(image_size, image_size, 3))
base_model.trainable = False

model_vgg = Sequential([
    base_model,
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(len(labels), activation='softmax')
])

model_vgg.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history_vgg = model_vgg.fit(
    X_train, Y_train_onehot,
    epochs=25,
    batch_size=32,
    verbose=1
)

# === STEP 4: EVALUATION ON TEST SET ===
test_loss_vgg, test_accuracy_vgg = model_vgg.evaluate(X_test, Y_test_onehot, verbose=2)
print(f"VGG16 Test Loss: {test_loss_vgg:.4f}")
print(f"VGG16 Test Accuracy: {test_accuracy_vgg * 100:.2f}%")

pred_vgg = np.argmax(model_vgg.predict(X_test), axis=1)
actual_label_vgg = np.argmax(Y_test_onehot, axis=1)

labels_final = ['AD', 'CN', 'MCI']
print(classification_report(actual_label_vgg, pred_vgg, target_names=labels_final))
```

```
59/59 ━━━━━━━━━━━━━━━━━━━━ 523s 9s/step - accuracy: 0.8720 - loss: 0.3056
```

```
59/59 ━━━━━━━━━━━━━━━━━━━━ 557s 9s/step - accuracy: 0.9054 - loss: 0.2211
Epoch 10/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 568s 9s/step - accuracy: 0.9291 - loss: 0.1951
Epoch 11/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 559s 9s/step - accuracy: 0.9581 - loss: 0.1209
Epoch 12/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 524s 9s/step - accuracy: 0.9643 - loss: 0.0920
Epoch 13/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 522s 9s/step - accuracy: 0.9660 - loss: 0.0826
Epoch 14/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 564s 9s/step - accuracy: 0.9629 - loss: 0.1020
Epoch 15/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 529s 9s/step - accuracy: 0.9521 - loss: 0.1264
Epoch 16/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 526s 9s/step - accuracy: 0.9399 - loss: 0.1439
Epoch 17/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 560s 9s/step - accuracy: 0.9546 - loss: 0.1208
Epoch 18/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 563s 9s/step - accuracy: 0.9309 - loss: 0.1569
Epoch 19/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 524s 9s/step - accuracy: 0.9573 - loss: 0.1097
Epoch 20/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 523s 9s/step - accuracy: 0.9647 - loss: 0.1016
Epoch 21/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 520s 9s/step - accuracy: 0.9384 - loss: 0.1523
Epoch 22/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 563s 9s/step - accuracy: 0.9388 - loss: 0.1425
Epoch 23/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 562s 9s/step - accuracy: 0.9367 - loss: 0.1520
Epoch 24/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 521s 9s/step - accuracy: 0.9539 - loss: 0.1046
Epoch 25/25
59/59 ━━━━━━━━━━━━━━━━━━━━ 561s 9s/step - accuracy: 0.9419 - loss: 0.1292
15/15 - 129s - 9s/step - accuracy: 0.9787 - loss: 0.1418
VGG16 Test Loss: 0.1418
VGG16 Test Accuracy: 97.87%
15/15 ━━━━━━━━━━━━━━━━━━━━ 131s 9s/step
              precision    recall  f1-score   support

          AD       1.00      0.94      0.97       107
          CN       0.99      0.98      0.98       183
         MCI       0.96      1.00      0.98       179

    accuracy                           0.98       469
   macro avg       0.98      0.97      0.98       469
weighted avg       0.98      0.98      0.98       469
```

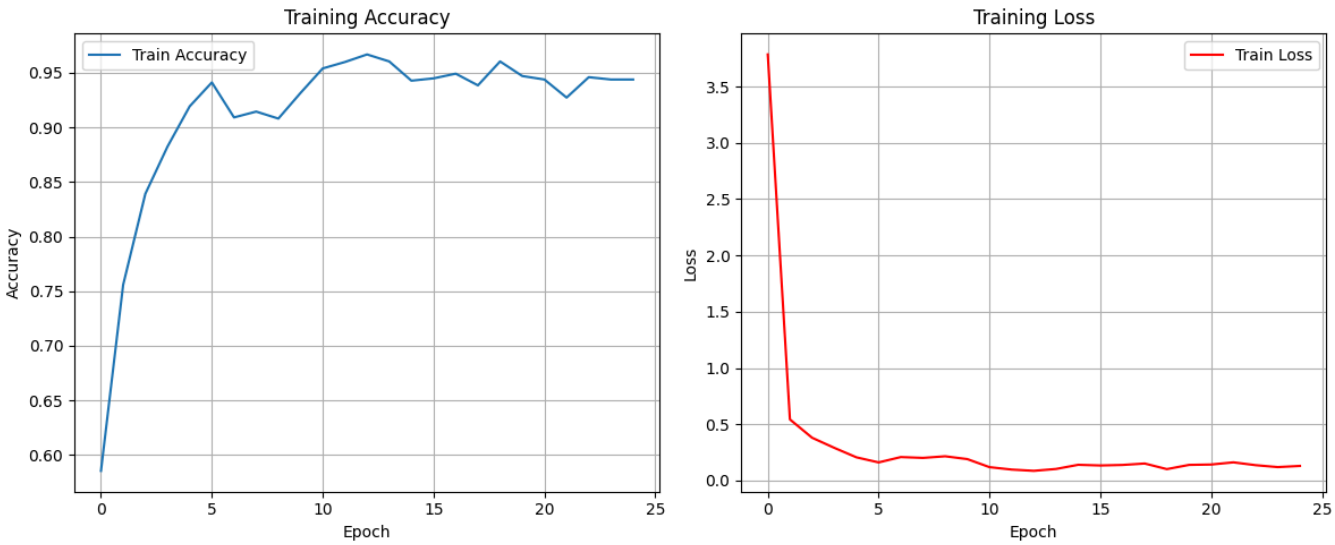
Start coding or generate with AI.

```
cm_vgg = confusion_matrix(actual_label_vgg, pred_vgg)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_vgg, cmap='Purples', annot=True, fmt='d', xticklabels=labels_final, yticklabels=labels_final)
plt.title("Confusion Matrix (VGG16)")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.savefig('vgg16_confusion_purples.png', dpi=300)
plt.show()
```




```
plt.figure(figsize=(12, 5))

# Accuracy
plt.subplot(1, 2, 1)
plt.plot(history_vgg.history['accuracy'], label='Train Accuracy')
plt.title('Training Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()

# Loss
plt.subplot(1, 2, 2)
plt.plot(history_vgg.history['loss'], label='Train Loss', color='red')
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.savefig('training_accuracy_loss.png', dpi=300)
plt.show()

# === STEP 7: K-FOLD CROSS VALIDATION ON TRAINING DATA ===
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
fold_no = 1
cv_accuracies = []

for train_idx, val_idx in kfold.split(X_train):
    print(f"\nTraining on Fold {fold_no}...")
    x_train_fold, x_val_fold = X_train[train_idx], X_train[val_idx]
    y_train_fold, y_val_fold = Y_train_onehot[train_idx], Y_train_onehot[val_idx]

    base_model_cv = VGG16(weights='imagenet', include_top=False, input_shape=(image_size, image_size, 3))
    base_model_cv.trainable = False

    model_cv = Sequential([
        base_model_cv,
        Flatten(),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(len(labels), activation='softmax')
    ])

    model_cv.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

    model_cv.fit(x_train_fold, y_train_fold, epochs=10, batch_size=32, verbose=0)
    val_loss, val_acc = model_cv.evaluate(x_val_fold, y_val_fold, verbose=0)
    print(f"Fold {fold_no} Accuracy: {val_acc * 100:.2f}%")
    cv_accuracies.append(val_acc * 100)
```

```
    fold_no += 1

print(f"\nAverage Cross-Validation Accuracy: {np.mean(cv_accuracies):.2f}% ± {np.std(cv_accuracies):.2f}%")
```




```
Training on Fold 1...
```

```
# === STEP 8: PLOT CROSS-VALIDATION ACCURACY PER FOLD ===
plt.figure(figsize=(8, 5))
folds = list(range(1, len(cv_accuracies) + 1))
plt.plot(folds, cv_accuracies, marker='o', linestyle='-', color='green', label='Fold Accuracy')
plt.axhline(y=np.mean(cv_accuracies), color='r', linestyle='--', label=f'Mean Accuracy: {np.mean(cv_accuracies):.2f}%')
plt.title("Cross-Validation Accuracy per Fold")
plt.xlabel("Fold")
plt.ylabel("Validation Accuracy (%)")
plt.xticks(folds)
plt.grid(True)
plt.legend()
plt.tight_layout()
```